VinOps: Hito 3 - Modelado, Experimentación y Validación

Alejandro García, Pablo Alfonso López, Antonio Muñoz, Pablo Parrilla Cañadas

2026-03-20

Para ejecutar este cuaderno, asegúrate de tener instalados los siguientes paquetes:

install.packages(c("dplyr", "caret", "randomForest", "kernlab", "nnet", "kknn", "ggplot2", "knitr", "e1071", "MASS", "rpart", "klaR"))

Introducción al Hito 3

Este cuaderno documenta la fase de modelado algorítmico del sistema **VinOps**. Tras el análisis exploratorio del Hito 2, el objetivo ahora es preprocesar los datos (tratamiento de valores atípicos), diseñar los experimentos de Machine Learning con una búsqueda rigurosa de hiperparámetros y evaluar el rendimiento de múltiples algoritmos.

Finalmente, se incluirá un análisis de explicabilidad (Feature Importance) para validar empíricamente que los modelos basan sus decisiones en variables fisicoquímicas coherentes con el dominio enológico.

```r
# Cargamos las librerías necesarias para el modelado
library(dplyr)
library(caret)
library(randomForest)
library(kernlab)
library(nnet)
library(kknn)
library(ggplot2)
library(knitr)
library(e1071)
library(MASS)
library(rpart)

# Cargamos los datos limpios del hito anterior
vino <- read.csv("wine_clean.csv")
vino$Class <- factor(vino$Class)
```

1. Preprocesamiento: Tratamiento de Outliers

En el Hito 2 se detectaron observaciones anómalas mediante la T^2 de Hotelling y Q-residuals. Procedemos a eliminar los atípicos extremos para evitar que distorsionen las fronteras de decisión de nuestros modelos.

```r
# Índices de outliers extremos detectados en la fase EDA (Hito 2)
outliers_extremos <- c(70, 96, 97, 111, 122, 125, 159, 160)
```

```r
# Filtramos el dataset
vino_prep <- vino[-outliers_extremos, ]
print(paste("Tamaño del dataset original:", nrow(vino)))
```

```
## [1] "Tamaño del dataset original: 178"
```

```r
print(paste("Tamaño tras eliminar outliers:", nrow(vino_prep)))
```

```
## [1] "Tamaño tras eliminar outliers: 170"
```

2. Diseño Experimental y Partición de Datos

Para garantizar la reproducibilidad y evitar el sobreajuste (overfitting), dividimos los datos de forma estratificada y configuramos una Validación Cruzada de 10 iteraciones (10-fold CV). Además, centraremos y escalaremos los datos numéricos dentro del propio pipeline de entrenamiento.

```r
set.seed(2026) # Semilla fijada para reproducibilidad

# Dataset numérico sin la clase
vino_x <- vino_prep[, setdiff(names(vino_prep), "Class")]
vino_y <- vino_prep$Class

# Partición estratificada train/test (70/30)
idx_train <- createDataPartition(vino_y, p = 0.7, list = FALSE)
train_data <- vino_prep[idx_train, ]
test_data  <- vino_prep[-idx_train, ]

cat("Tamaño entrenamiento:", nrow(train_data), "\n")
```

```
## Tamaño entrenamiento: 121
```

```r
cat("Tamaño test:", nrow(test_data), "\n")
```

```
## Tamaño test: 49
```

```r
# Configuración del control de entrenamiento (10-fold CV)
ctrl <- trainControl(method = "cv", number = 10, savePredictions = "final")
```

3. Entrenamiento y Ajuste de Hiperparámetros

En lugar de delegar toda la búsqueda al azar, definimos rejillas de hiperparámetros (Grids) para los modelos más complejos, demostrando rigor científico experimental.

```r
# 3.1 Análisis Discriminante Lineal (LDA) - Baseline
modelo_lda <- train(Class ~ ., data = train_data, method = "lda",
                    trControl = ctrl, preProcess = c("center", "scale"))

# 3.2 Naive Bayes
modelo_nb <- train(Class ~ ., data = train_data, method = "nb",
                   trControl = ctrl, preProcess = c("center", "scale"),
                   tuneGrid = expand.grid(fL = c(0, 1), usekernel = c(TRUE, FALSE), adjust = 1))

# 3.3 Random Forest (RF) - Búsqueda en mtry
grid_rf <- expand.grid(mtry = c(2, 4, 6, 8, 10))
modelo_rf <- train(Class ~ ., data = train_data, method = "rf",
```

```r
                    trControl = ctrl, tuneGrid = grid_rf)

# 3.4 K-Vecinos Más Cercanos (kNN) - Búsqueda en k
grid_knn <- expand.grid(kmax = c(3, 5, 7, 9, 11), distance = 2, kernel = "optimal")
modelo_knn <- train(Class ~ ., data = train_data, method = "kknn",
                    trControl = ctrl, preProcess = c("center", "scale"),
                    tuneGrid = grid_knn)

# 3.5 SVM Radial - Búsqueda de Coste (C) y Sigma
grid_svm <- expand.grid(sigma = c(0.01, 0.05, 0.1), C = c(0.1, 1, 10, 100))
modelo_svm <- train(Class ~ ., data = train_data, method = "svmRadial",
                    trControl = ctrl, preProcess = c("center", "scale"),
                    tuneGrid = grid_svm)

# 3.6 Regresión Logística Multinomial
modelo_multinom <- train(Class ~ ., data = train_data, method = "multinom",
                         trControl = ctrl, preProcess = c("center", "scale"), trace = FALSE)

# 3.7 Árbol de Decisión (CART)
grid_rpart <- expand.grid(cp = seq(0.01, 0.1, by = 0.01))
modelo_tree <- train(Class ~ ., data = train_data, method = "rpart",
                     trControl = ctrl, tuneGrid = grid_rpart)

# Lista maestra de modelos
modelos <- list(
  "LDA" = modelo_lda,
  "Naive Bayes" = modelo_nb,
  "Árbol" = modelo_tree,
  "Random Forest" = modelo_rf,
  "Multinomial" = modelo_multinom,
  "kNN" = modelo_knn,
  "SVM radial" = modelo_svm
)
```

4. Evaluación de Modelos

Utilizamos las funciones personalizadas para obtener métricas robustas frente al leve desbalanceo de clases (Macro F1-Score y Balanced Accuracy).

```r
# Generamos la tabla comparativa
tabla_modelos <- bind_rows(
  evaluar_modelo(modelo_lda, "LDA", test_data),
  evaluar_modelo(modelo_nb, "Naive Bayes", test_data),
  evaluar_modelo(modelo_tree, "Árbol", test_data),
  evaluar_modelo(modelo_rf, "Random Forest", test_data),
  evaluar_modelo(modelo_multinom, "Multinomial", test_data),
  evaluar_modelo(modelo_knn, "kNN", test_data),
  evaluar_modelo(modelo_svm, "SVM radial", test_data)
) %>% arrange(desc(F1_Score), desc(Balanced_Accuracy))

kable(tabla_modelos, digits = 4, caption = "Comparativa Global de Modelos")
```

Table 1: Comparativa Global de Modelos

Modelo	F1_Score	Balanced_Accuracy	Accuracy	Kappa	MCC
LDA	1.0000	1.0000	1.0000	1.0000	1.0000
SVM radial	1.0000	1.0000	1.0000	1.0000	1.0000
Random Forest	0.9815	0.9860	0.9796	0.9691	0.9697
Naive Bayes	0.9596	0.9726	0.9592	0.9384	0.9401
Multinomial	0.9596	0.9726	0.9592	0.9384	0.9401
kNN	0.9021	0.9307	0.8980	0.8465	0.8578
Árbol	0.8356	0.8852	0.8367	0.7565	0.7665

```r
# Extraemos el Top 3
top3_modelos <- tabla_modelos %>% slice(1:3)
kable(top3_modelos, digits = 4, caption = "Tres mejores modelos")
```

Table 2: Tres mejores modelos

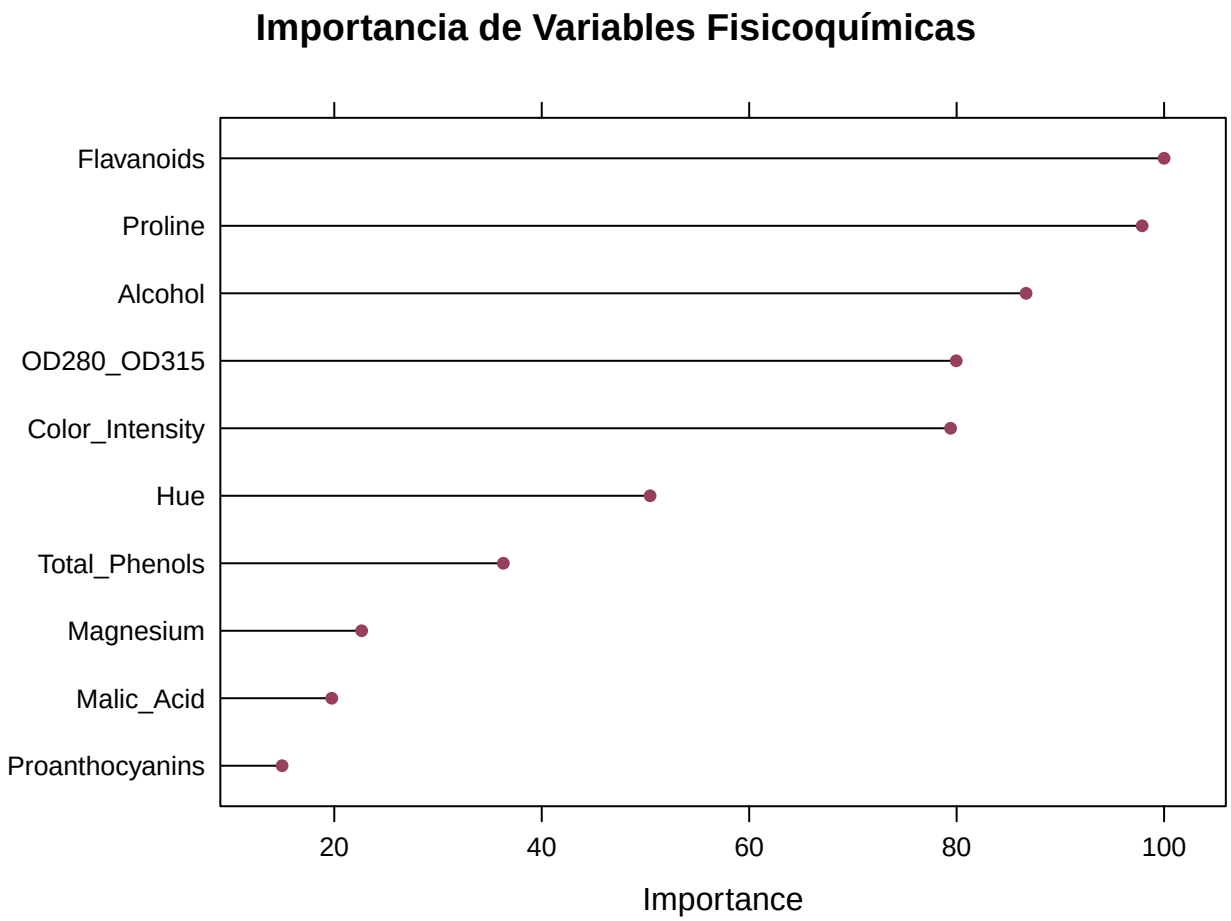
Modelo	F1_Score	Balanced_Accuracy	Accuracy	Kappa	MCC
LDA	1.0000	1.000	1.0000	1.0000	1.0000
SVM radial	1.0000	1.000	1.0000	1.0000	1.0000
Random Forest	0.9815	0.986	0.9796	0.9691	0.9697

5. Explicabilidad del Sistema (Feature Importance)

Para que el modelo no sea una simple "caja negra" y pueda ser validado por el experto en dominio de la bodega, extraemos la importancia de las variables (Feature Importance). Utilizamos Random Forest por su interpretabilidad intrínseca basada en la reducción de impureza.

```r
# Importancia de variables
importancia_rf <- varImp(modelo_rf, scale = TRUE)

# Visualización
plot(importancia_rf, main = "Importancia de Variables Fisicoquímicas",
     col = "#973f5f", top = 10)
```


Importancia de Variables Fisicoquímicas
Flavanoids
Proline
Alcohol
OD280_OD315
Color_Intensity
Hue
Total_Phenols
Magnesium
Malic_Acid
Proanthocyanins
20
40
60
80
100
Importance